



# PROGRAMACIÓN iii

# Branches

- Ya sabemos que Git puede manejar varias versiones de un mismo proyecto al mismo tiempo. Pero, ¿dónde guarda cada una de estas versiones? La respuesta está en las ramas o branches, en inglés.
- En lugar de tener carpetas amontonadas con nombres como "Versión 1", "Versión 2" o "Versión Final", Git utiliza ramas. Estas nos dan la posibilidad de intercalar entre distintos estados del proyecto de forma sencilla. Podemos tener una rama para cómo estaba el código antes de un cambio, otra con el cambio realizado, y si algo deja de funcionar, podemos volver al punto anterior simplemente cambiando de rama.

# Branches

- También son sumamente útiles cuando trabajamos en distintos entornos como el de desarrollo, pruebas o producción, permitiéndonos tener ramas que simulen o contengan los archivos correspondientes a cada uno de esos ambientes.
- En su definición formal, las ramas pueden definirse como “punteros” móviles hacia cada uno de los cambios o versiones que armamos en nuestro proyecto. Cada rama representa una línea independiente de desarrollo.

# Branches

- Git cuenta con una rama principal, históricamente llamada master, hoy frecuentemente llamada main; de la cual parten todas las demás ramas.
- Siempre partimos de esta rama principal. Si queremos realizar nuevos cambios o agregar funcionalidades, podemos crear una rama nueva, trabajar allí de forma aislada e incorporar esas novedades a la rama principal más adelante. Mientras tanto, otra persona puede estar trabajando simultáneamente en su propia rama y, eventualmente, ambos podrán unificar su trabajo en la rama principal mediante distintos comandos. En conclusión, las ramas nos permiten a varias personas trabajar en simultáneo sin “pisarnos” el código.



# Branches

- Imagen ilustrativa del trabajo en ramas:



# Branches



Para entender esto de forma práctica, ubíquense en el último repositorio local desarrollado en clases, donde vimos el uso de git clone y git pull. Abran la consola de Git Bash allí y sigan estos pasos:

1. Para identificar en qué rama estamos, ingresen el siguiente comando:

*git branch*

Este comando nos muestra una lista de todas las ramas locales existentes. La rama en la que nos encontramos actualmente aparecerá con un asterisco y resaltada en verde.



# Branches

2. Para crear nuestra primera línea de desarrollo paralela, ingresaremos:

*git branch rama\_1*

Si colocamos nuevamente el comando *git branch*, veremos las ramas existentes, pero notaremos que seguimos ubicados en la principal.



# Branches

Reglas de sintaxis obligatorias y buenas prácticas para nombrar ramas:

- Sin espacios: No se pueden usar espacios en blanco. Separen las palabras con guiones medios o bajos.
- Sin caracteres especiales: Eviten tildes, la letra ñ, y símbolos como ~ ^ : ? \* [ \
- Uso de minúsculas: Aunque Git distingue entre mayúsculas y minúsculas, por convención y para evitar confusiones, se recomienda escribir todo en minúsculas.
- Ser descriptivos: El nombre debe indicar claramente el propósito de la rama. Por ejemplo: `corregir_error_login` o `agregar_menu`.



# Branches



¿Qué pasa si colocamos erróneamente el nombre de una rama o queremos mejorarlo? Podemos cambiarlo mediante el comando: `git branch -m nombre_viejo nombre_nuevo`

3. Realicen la prueba colocando lo siguiente:

`git branch -m rama_1 primera_rama`

Si ejecutan nuevamente `git branch` verán que la rama adoptó el último nombre que le asignamos.



# Branches



4. Para efectivamente ubicarnos y empezar a trabajar dentro de la otra rama, ingresaremos el comando: *git checkout nombre\_de\_la\_rama*. En nuestra práctica:

*git checkout primera\_rama*

Colocamos *git branch* nuevamente y veremos resaltado en verde que ahora sí estamos ubicados en nuestra nueva rama. Todos los cambios que hagamos de ahora en adelante se guardarán solo aquí.



# Branches



5. Si queremos eliminar una rama de nuestro repositorio, lo realizamos mediante el comando: *git branch -d nombre\_de\_la\_rama*. Por seguridad, Git exige que estemos ubicados fuera de la rama que queremos borrar. Aun así, intenten ingresar el comando para visualizar el sistema de protección:

*git branch -d primera\_rama*

Deberían visualizar un mensaje de error advirtiéndolo que no pueden borrar la rama actual.





# Branches



Bien, realicemos el borrado de forma correcta. Para ello, primero volvamos a la rama principal:

*git checkout master*

Si su rama principal se llama main, usen *git checkout main*

Y luego procedemos con la eliminación:

*git branch -d primera\_rama*

Por último, coloquen el comando *git branch* para comprobar que el procedimiento se llevó a cabo correctamente y la rama desapareció de la lista.







**Instituto Superior  
Del Milagro**  
NIVEL SUPERIOR Terciario Nº 8207